Generative AI - Question Bank

LLMs, RAG, prompting, and how it maps to your work and SAP

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 8 of 8

Contents

How to use this

Generative AI is on your resume three ways: the **"Generative AI with Large Language Models"** certificate, the **FloatChat** project (LangChain + Mistral 7B + RAG), and the AI-assisted **Glass Automation** work. On top of that, your trainer said to have a general idea of **SAP's generative-AI workflows** (Joule). So this is a high-probability topic, and you have real experience to speak from - use it.

Each item is a likely question with a clear, correct answer at your level. Two habits to carry:

- **Speak from FloatChat wherever possible.** It's a real, working generative-AI system you built. "In FloatChat I did exactly this" beats any textbook definition.
- **Bridge to SAP.** FloatChat (natural language to real data) is the same pattern as SAP's Joule. Saying that shows you connect your work to their world.

The questions marked *(link:)* have a natural tie-in to your projects or to SAP.

Foundations

What is Generative AI?

AI that **creates new content** - text, code, images - rather than just classifying or predicting from fixed options. Traditional machine learning might label an email as spam or not; generative AI writes a new email. It learns patterns from huge amounts of data and generates fresh output that fits those patterns.

Generative AI vs traditional machine learning - what's the difference?

- **Traditional ML** is mostly **discriminative**: given input, predict a label or number (spam/not, price, yes/no).
- **Generative AI** produces **new, open-ended content** (a paragraph, a SQL query, an image).

Both learn from data; the difference is that one chooses among known answers and the other creates. *(link: FloatChat generates brand-new SQL from a plain-English question - that's generation, not classification.)*

What is an LLM?

A **Large Language Model** - an AI model trained on massive amounts of text that can understand and generate human language. It works by predicting the next **token** (word piece) over and over, which, at scale, produces coherent answers, code, and reasoning. *(link: I used Mistral 7B, an LLM, in FloatChat.)*

What is a foundation model? What is pretraining?

A **foundation model** is a large model **pretrained** on broad, general data, which can then be adapted to many tasks. **Pretraining** is that first, expensive phase of learning general language patterns from enormous text. After pretraining, the same model can be prompted or fine-tuned for specific jobs - hence "foundation."

What is a transformer? What is attention (in simple terms)?

The **transformer** is the neural-network architecture behind modern LLMs. Its key idea is **attention**: when processing a word, the model **weighs how much every other word matters** to it, so it captures context and relationships across a whole sentence or document. You don't need the math - just: "transformers use attention to focus on the relevant parts of the input when generating each word."

The building blocks

What is a token?

The unit an LLM actually reads and generates - roughly a word or word-piece (about 3 to 4 characters of English on average). "Tokenization" is splitting text into tokens. Models are priced and limited by tokens, so token count matters for cost and speed.

What is a context window?

The **maximum amount of text (in tokens) a model can consider at once** - the prompt plus its answer. If a conversation exceeds it, the oldest parts drop off. This is why RAG matters: you can't paste an entire database into the context, so you retrieve only the relevant pieces.

What do "parameters" mean, like in Mistral 7B?

Parameters are the model's learned internal values (weights). "7B" means **7 billion parameters**. More parameters generally means more capability but also more cost, memory, and slower inference. *(link: I chose a 7B model in FloatChat deliberately - big enough to be capable, small enough for predictable cost and latency.)*

What is an embedding?

A way to turn text (or an image) into a **list of numbers (a vector) that captures its meaning**. Texts with similar meaning get similar vectors, so you can measure similarity mathematically. Embeddings are the foundation of semantic search and RAG.

What is a vector database?

A database that stores embeddings and lets you search by **meaning** - "find the items whose vectors are closest to this one" - rather than exact keyword match. *(link: FloatChat used Supabase for vector storage to retrieve similar past queries as context.)*

RAG and prompting (your strongest area)

What is RAG, and why use it?

Retrieval-Augmented Generation. Before the model answers, you **retrieve relevant information** (from a vector database or documents) and feed it into the prompt, so the answer is **grounded in real data** instead of the model's fuzzy memory. It reduces hallucination and lets the model use up-to-date or private information it was never trained on. *(link: FloatChat uses RAG - it retrieves similar earlier queries and the schema so the model generates correct SQL.)*

What is prompt engineering?

The craft of **writing the input to get the best output** - being clear, giving context, showing examples, and constraining the format. In practice it's the cheapest, fastest way to improve an LLM's behaviour before reaching for anything heavier. *(link: in FloatChat I used schema-aware prompting - telling the model the exact table structure so its SQL is valid.)*

Zero-shot, few-shot, and chain-of-thought prompting?

- **Zero-shot:** just ask, no examples.
- **Few-shot:** include a few examples of the task in the prompt so the model follows the pattern.
- **Chain-of-thought:** ask the model to reason step by step, which improves accuracy on harder problems.

What is a system prompt?

A hidden instruction that sets the model's **role and rules** for the whole conversation ("You are a helpful SQL assistant; only output valid PostgreSQL"). It shapes every answer without the user seeing it.

What is grounding?

Tying the model's output to **verified, real information** (via RAG or tool calls) so it can't just make things up. Grounding is what turns a chatty model into a trustworthy one. *(link: FloatChat is grounded - it runs generated SQL against the real database and validates it before showing results.)*

Reliability and limitations

What is hallucination? How do you reduce it?

When an LLM produces something **fluent but false** - a made-up fact, a wrong citation, invalid code - stated confidently. Reduce it with: **RAG** (ground answers in real data), **validation** (check the output before using it), **lower temperature**, and **constraining** the task with a clear prompt. *(link: FloatChat validates every generated query before running it, so a bad guess never reaches the database.)*

What is temperature (and top-p)?

Temperature controls randomness. **Low temperature** (near 0) makes output focused and deterministic - good for code and SQL. **High temperature** makes it more creative and varied - good for brainstorming. **Top-p** is a related setting that limits choices to the most probable tokens. *(link: for FloatChat's SQL generation you want low temperature - you need correct, not creative.)*

What are the main limitations of LLMs? When would you NOT use one?

- They can **hallucinate**, and they don't truly "know" - they predict likely text.
- They have a **knowledge cutoff** and a limited **context window**.
- They can be **expensive and slow** at scale, and **non-deterministic**.
- They can reflect **bias** from training data.

Don't use an LLM when a simple rule, a database query, or classic ML would be cheaper and more reliable. Good engineering is knowing when *not* to reach for AI - which is itself a great thing to say.

Adapting models and building systems

Fine-tuning vs prompting vs RAG - when do you use each?

Three ways to make a model do what you want, cheapest first:

- **Prompting** - just write better instructions/examples. Fast, no training. Try this first.
- **RAG** - give the model your data at query time via retrieval. Best when the model needs **your specific or fresh knowledge**.
- **Fine-tuning** - further-train the model on your examples to change its behaviour/style. Most effort and cost; use when prompting and RAG aren't enough.

(link: FloatChat used prompting + RAG, not fine-tuning - the cheaper, faster path that was good enough, which was the right engineering call.)

What is an AI agent?

An LLM that can **take actions**, not just chat - it can call tools, run code, query a database, or search the web, and loop until a goal is met. Instead of only answering, it *does*. (SAP's Joule increasingly works this way, acting across SAP systems.)

What is LangChain?

A framework for **wiring LLM applications together** - chaining the steps of take a question, add context, call the model, validate, return, and connecting to tools, vector stores, and retrieval. *(link: I used LangChain to orchestrate FloatChat's pipeline.)*

Open-source vs closed-source models (Mistral vs GPT-4)?

- **Open models (Mistral, Llama)** - you can self-host, control cost and data privacy, and customise. Often smaller.
- **Closed models (GPT-4, Claude)** - accessed via API, usually more capable, but with per-call cost and less control.

(link: I picked open Mistral 7B for FloatChat for predictable cost and latency per message, and made it reliable with good engineering rather than raw model size.)

What is quantization (briefly)?

Shrinking a model by storing its numbers at lower precision (e.g. 4-bit instead of 16-bit), so it runs faster and on cheaper hardware, with a small accuracy trade-off. Good to know the term; you won't be quizzed deeply.

Natural-language-to-SQL and the SAP bridge

What is natural-language-to-SQL, and how does FloatChat do it?

It's turning a plain-English question into a database query the model writes for you. In FloatChat: the user asks in English, the LLM (guided by the table schema and retrieved examples) generates SQL, the query is **validated before running**, it executes against PostgreSQL, and the results render as maps and charts. **The key engineering point:** I constrained the model (schema-aware prompts, validation, RAG) so a 7B model produces reliable SQL - reliability came from the system design, not just the model.

How does this connect to SAP's Business AI / Joule?

Directly. **Joule** is SAP's generative-AI copilot: you ask business questions in plain language and it acts across SAP's data and applications. That's the **same pattern as FloatChat** - natural language in, grounded answers from real data out - just on enterprise business data instead of ocean data. **SAP Business AI** is the broader idea of embedding this into real business processes. Saying "I've built a small version of what Joule does" is a genuinely strong line.

What is SAP's approach to generative AI, in one breath?

AI embedded **into business processes** and grounded in a company's **real business data**, delivered mainly through **Joule** (the copilot) and built on **SAP BTP's AI services**. Not AI for novelty - AI that does real work like drafting orders, summarising reports, and answering business questions.

The GenAI cheat-sheet

The one-line versions to have ready:

Term	One-line answer
Generative AI	AI that creates new content, not just labels it
LLM	Large model that predicts the next token to generate language
Token	The word-piece unit a model reads and generates
Context window	Max tokens a model can consider at once
Embedding	Text turned into a meaning-capturing vector
Vector DB	Stores embeddings, searches by meaning
RAG	Retrieve real context first, then generate - grounds the answer
Prompt engineering	Writing input well to get better output
Temperature	Randomness dial; low = focused, high = creative
Hallucination	Fluent but false output; reduce with RAG + validation
Fine-tuning	Further-training a model on your data (last resort)
Agent	An LLM that takes actions and uses tools
Joule	SAP's generative-AI copilot - same pattern as FloatChat

Your closing move on any GenAI question: land it on FloatChat, then bridge to Joule. "I built exactly this pattern in FloatChat - natural language to real data - which is the same thing SAP does with Joule at enterprise scale." That single sentence proves experience *and* company research at once.